

SAMSUDEEN SABEER RAHMAN - 63920

MEKA VENKATA SRAVANTH BABU-63919

SAKTHIVEL BALAKUMAR - 63957

ELECTRONIC MICROSYSTEMS

IE.2409

LAB1_SOC_NIOS_DE10_V18

5.3 Interact with LEDS:

Replacing the entire program of “Hello_world.c” file with the following program (copy it from MOODLE file "Test_Leds.c"):

```
/*
 * Test Leds
 *
 */
#include "system.h"
#include <stdio.h>
#include "alt_types.h"
#include "altera_avalon_pio_regs.h"
void Tempo(int nbwait);
#define DELAY 200000
int main()
{

    printf("-----\n");
    printf("Welcome in the wonderful world of NIOS II\n");
    printf("-----\n\n");
    while (1) {
        IOWR_ALTERA_AVALON_PIO_DATA(LEDPIO_BASE, 0xff);
        Tempo(DELAY);
    }
}
```

```

        IOWR_ALTERA_AVALON_PIO_DATA(LEDPIO_BASE, 0x00);
        Tempo(DELAY);
    }
    return 0;
}

void Tempo(int nbwait)
{
    while (nbwait > 0) {
        nbwait--;
    }
}

```

This program is a simple LED blinking program for an NIOS II processor. This program uses a delay function to control the timing of the LED toggling.

The output will be :

```

-----
Welcome in the wonderful world of NIOS II
-----

```

Q1)After changing the delay to ½ seconds flash period.

```

#include <stdio.h>
#include "system.h"
#include "alt_types.h"
#include "altera_avalon_pio_regs.h"
void Tempo(int nbwait);
#define DELAY 6250000 // It is for 0.5-second delay at 50MHz clock
int main()
{
    printf("-----\n");
    printf("Welcome in the wonderful world of NIOS II\n");
    printf("-----\n\n");

    while (1) {

```

```

IOWR_ALTERA_AVALON_PIO_DATA(LEDPIO_BASE, 0xff); // Turn LEDs ON
Tempo(DELAY);
IOWR_ALTERA_AVALON_PIO_DATA(LEDPIO_BASE, 0x00); // Turn LEDs OFF
Tempo(DELAY);
}
return 0;
}

```

```

void Tempo(int nbwait)
{
    while (nbwait > 0) {
        nbwait--;
    }
}

```

- If delay is increased ,the blinking period increases, so it will make it more visible. If delay decreased ,faster blinking occurs.

Q2) When it works, modify the file to operate a binary counter on the 4 LEDs. Choose a period of 2 seconds or more to clearly see the count.

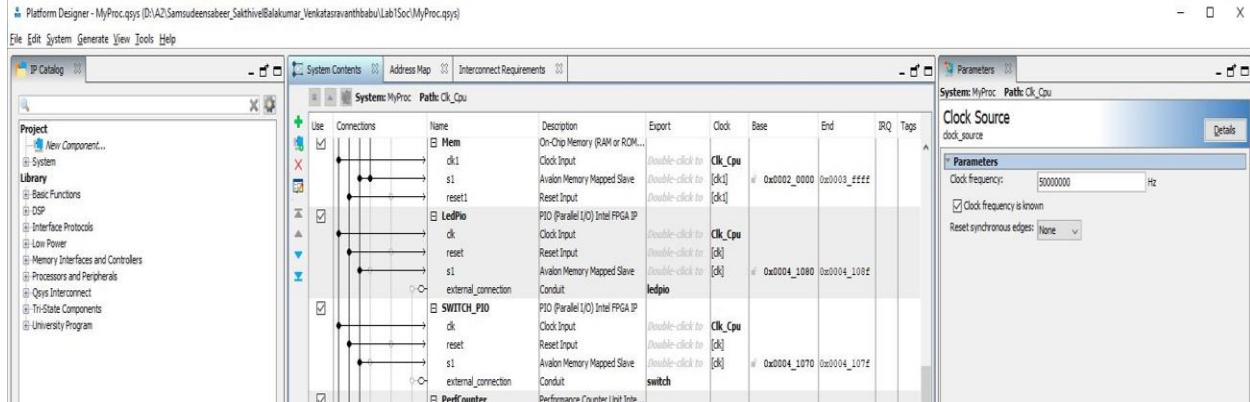
```
#define DELAY 25000000
```

- Yes this time the LED will blink very slowly.

5.4 Interact with Switches:

Using the documentation the pin numbers of 4 switches are identified as

<i>Signal Name</i>	<i>FPGA Pin No.</i>	<i>Description</i>	<i>I/O Standard</i>
SW[0]	PIN_AB30	Slide Switch[0]	Depend on JP3
SW[1]	PIN_Y27	Slide Switch[1]	Depend on JP3
SW[2]	PIN_AB28	Slide Switch[2]	Depend on JP3
SW[3]	PIN_AC30	Slide Switch[3]	Depend on JP3



- A PIO with 4 bits in INPUT mode is added and it is renamed as SWITCH_PIO.

Following steps are followed ,then saved and generated.

Then symbols are updated in the schematic window.

Q3) Create a program which display the state of the switches on the screen and copy it on the LEDs

Program:

```
#include <stdio.h>

#include "system.h"

#include "alt_types.h"

#include "altera_avalon_pio_regs.h"

void Tempo(int nbwait);

#define DELAY 5000000 // Small delay to slow down screen updates

int main()
{
    printf("-----\n");
    printf("Reading Switches and Displaying on LEDs\n");
    printf("-----\n\n");

    int value;

    while (1) {
```

```

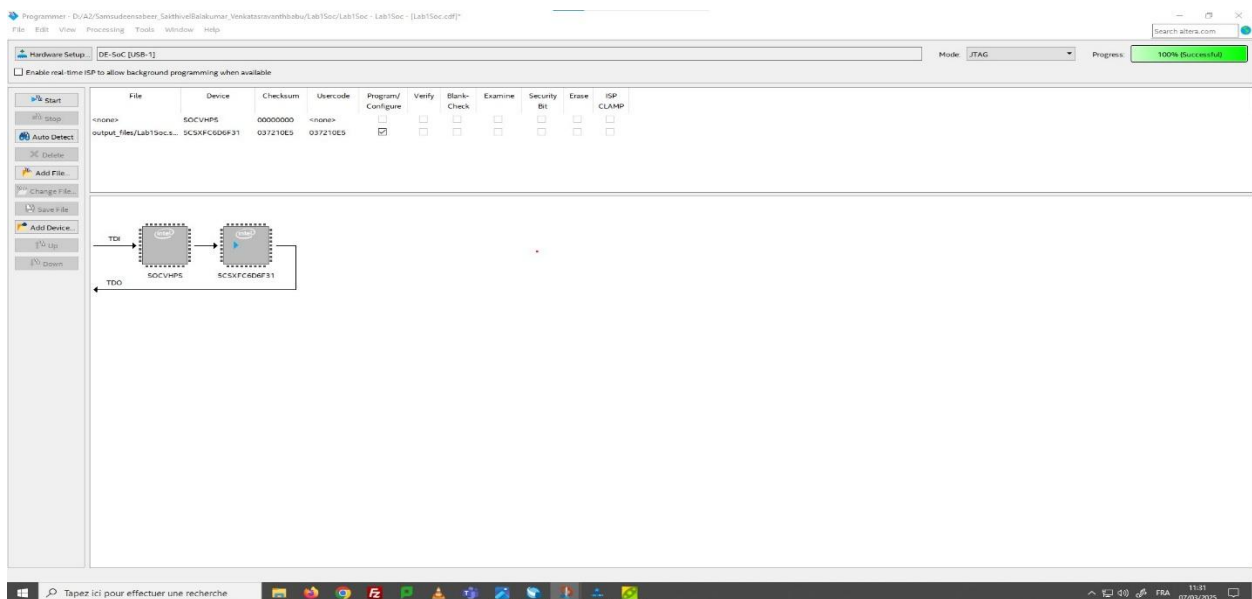
    value = IORD_ALTERA_AVALON_PIO_DATA(SWITCH_PIO_BASE); // Read switches
    printf("Switch value: %d\n", value); // Display value on the screen
    IOWR_ALTERA_AVALON_PIO_DATA(LEDPIO_BASE, value); // Copy to LEDs
    Tempo(DELAY); // Small delay to slow down refresh rate
}
return 0;
}

void Tempo(int nbwait)
{
    while (nbwait > 0) {
        nbwait--;
    }
}

```

- Displaying the state of the switches on the screen means that reading the ON/OFF (1/0) status of switch.

5.5 Hardware part:



100% successfully the hardware part is processed.

- Basically a multiplier is added to any digital circuits to perform arithmetic operations, such as multiplying two numbers.

-

5.6 Software part:

The screenshot shows the Eclipse IDE interface with the following components:

- Project Explorer:** Displays the project structure for "eight_hello_world". It includes folders for "Binaries", "Includes", "Obj", and "Source Files".
- Main Editor:** Shows the assembly code for "hello_world.s". The code defines constants for DATA1 and DATA2, sets up registers, and implements a loop to calculate the product of two numbers (short_data_1 and short_data_2). It also includes performance measurement macros.
- Outline:** Provides a hierarchical view of the source files in the project.
- Console:** Displays the output of the program, which includes a welcome message and performance statistics for both soft and hard versions of the multiplication.

```

14 short my_hard_multiply(short_data_1, short_data_2);
15
16 #define DATA1 0x7F
17 #define DATA2 0x11
18
19 // #define DATA1 0x7F
20 // #define DATA2 0x3F
21
22
23
24 int main()
25 {
26     //-----
27
28     alt_u64 counter;   int n, time;   short result;
29
30
31     printf("\n\n");
32     printf("===== \n");
33     printf("Welcome in the wonderful world of HDIO II \n");
34     printf("===== \n\n");
35
36     // code to measure performance of soft_multiply
37     PERF_RESET(PERFCOUNTER_BASE);
38     PERF_START(MEASURING(PERFCOUNTER_BASE));
39     PERF_BEGIN(PERFCOUNTER_BASE, 1);
40     for (n = 0; n < 1000; n++) {
41         result = my_soft_multiply(DATA1, DATA2);
42     }
43     PERF_END(PERFCOUNTER_BASE, 1);
44 }
```

Output from the console:

```

=====
Welcome in the wonderful world of HDIO II
=====

--- Soft version : Number of cycles for 1000 loop = 503070
--- Soft version : Number of cycles for 1 loop = 503

--- Hard version : Number of cycles for 1000 loop = 188070
--- Hard version : Number of cycles for 1 loop = 188

+++ End Performance Measurement
```

Generally this program measures the execution time of hardware and software based multiplication on the NIOS II processor.

- Nios II /e is a resource efficient softcore processor without hardware multiplication, division.
- Nios II /f is a high performance softcore processor with hardware multiplication and division.

1) Do some measurements with the current processor to fill the following table:

NIOSIIe	Time soft_multiply	Time Hard_multiply
DATA1 = 5 DATA2 = 17	Q4) 583	Q5) 188
DATA1 = 127 DATA2 = 17	Q6) 583	Q7) 188

Q5) On your idea, why do we have different time depending on the data for soft_multiply? (think of the multiplication algorithm)... 2) Change the processor for a Nios IIs and do the same measurement.

2) Change the processor for a Nios IIs and do the same measurement.

NIOSIIs	Time soft_multiply	Time Hard_multiply
DATA1 = 5 DATA2 = 17	Q8) 400	Q9) 188
DATA1 = 127DATA2 = 17	Q10) 400	Q11) 188

6 Software improvement:

Go back to PLATFORM_DESIGNER, and change the processor model for a NIOS II/e.

Q11) Copy and explain the code used to multiply two numbers (inside soft_multiply()).

From the Test_Perf.c file the code is copied.

```
short my_soft_multiply(short sval_a, short sval_b)
//-----
//-----
{
    short result;
    result = sval_a * sval_b;
    return result;
}
```

Explanation: The function `my_soft_multiply` multiplies two short integers. The two values are `sval_a` and `sval_b`. Answer is stored in the `result`. Then it returns the result as short.

In the properties of the eclipse project the debug option is disabled and optimization level 3 is activated.

Q12) Run your previous code and fill the following array:

```

14 short my_hard_multiply(short data_a, short data_b);
15
16 #define DATA1 0x7F
17 #define DATA2 0x11
18
19 // #define DATA1 0x7F
20 // #define DATA2 0x3F
21
22
23
24 int main()
25 //-----
26 {
27     alt_u64 counter;    int n, time;    short result;
28
29
30     printf("\n\n");
31     printf("-----\n");
32     printf("++ Welcome in the wonderful world of NIOS II ++\n");
33     printf("-----\n");
34
35     // code to measure performance of soft_multiply
36     PERF_RESET(PERFCOUNTER_BASE);
37     PERF_START_MEASURING(PERFCOUNTER_BASE);
38     PERF_BEGIN(PERFCOUNTER_BASE, 1);
39     for (n = 0; n < 1000; n++) {
40         result = my_soft_multiply(DATA1, DATA2);
41     }
42     PERF_END(PERFCOUNTER_BASE, 1);

```

```

+-----+
+ Welcome in the wonderful world of NIOS II +
+-----+

--- Soft version : Number of cycles for 1000 loop = 23
--- Soft version : Number of cycles for 1 loop = 0

--- Hard version : Number of cycles for 1000 loop = 9
--- Hard version : Number of cycles for 1 loop = 0

+++ End   Performance Measurement

```

	Nb cycles to perform ONE multiplication		Time to perform 1000 multiplications xxx xxx xxx (add spaces to number easy reading)
Soft multiply()	Q12)	0	23
Hard multiply()	Q13)	0	9

Q13) Use the debugger to explain the result Modify the hardware to perform a division between a 16 bits number and an 8 bits number.

NIOSIIIF	Time soft_divide	Time Hard_divide
DATA1 = 45 DATA2 = 7	Q14) 10	Q15) 0
DATA1 = 345 DATA2 = 17	Q16) 2	Q17) 0
